

Semana 12: Laboratório de Injeção SQL e de Comandos

Segurança de Sistemas Informáticos, 2025-26

Licenciatura em Engenharia Informática, Universidade do Minho

Vítor Francisco Fonte, 2026-04-19

Neste trabalho laboratorial, vai explorar duas classes de vulnerabilidades de *injeção* numa aplicação Python (SQL injection e OS command injection), remediá-las usando *queries* parametrizadas e primitivas de execução de comandos mais seguras, e refletir sobre como a injeção se compara às vulnerabilidades de corrupção de memória exploradas na Semana 11. Os exercícios requerem apenas Python 3 (3.10 ou mais recente); não é necessário nenhum ambiente Linux x86-64 dedicado. Os alunos submetem um relatório em PDF através da plataforma de e-learning.

Objetivos de Aprendizagem

Ao completar este trabalho, os alunos serão capazes de:

1. Reconhecer e explorar uma vulnerabilidade de SQL injection numa aplicação Python.
2. Reconhecer e explorar uma vulnerabilidade de OS command injection numa aplicação Python.
3. Aplicar práticas de código seguro (*queries* parametrizadas, evitar a invocação do *shell*) para remediar vulnerabilidades de injeção.
4. Refletir sobre as causas comuns que unem as classes de corrupção de memória (Semana 11) e de injeção (Semana 12).

Configuração

Este trabalho requer apenas **Python 3** (3.10 ou mais recente). Pode executar tudo diretamente no sistema operativo do seu *host*: não é necessária qualquer máquina virtual ou *container*, e o *Guia de Configuração do Ambiente* que acompanha a Semana 11 não se aplica aqui.

- **macOS:** o Python 3 vem com as Xcode Command Line Tools ou pode instalar uma versão atual via Homebrew (`brew install python`).
- **Windows:** instale a partir de <https://www.python.org/downloads/> ou via o pacote *Python 3* da Microsoft Store.

- **Linux:** o `python3` está tipicamente pré-instalado. Caso contrário, instale através do gestor de pacotes da sua distribuição (por exemplo `sudo apt install python3` em Debian ou Ubuntu).

Verifique:

```
python3 --version # deve reportar Python 3.10 ou mais recente
```

Crie uma diretoria de trabalho para este trabalho:

```
mkdir -p ~/ssi-lab-inject && cd ~/ssi-lab-inject
```

A Aplicação Vulnerável

Crie um ficheiro chamado `noteapp.py` com o seguinte conteúdo:

```
#!/usr/bin/env python3
"""A deliberately vulnerable note-taking application."""

import os
import sqlite3
import sys

DB_FILE = "notes.db"

def init_db():
    """Create the notes table if it does not exist."""
    conn = sqlite3.connect(DB_FILE)
    conn.execute(
        "CREATE TABLE IF NOT EXISTS notes "
        "(id INTEGER PRIMARY KEY, title TEXT, body TEXT)"
    )
    conn.execute(
        "INSERT OR IGNORE INTO notes (id, title, body) VALUES "
        "(1, 'Welcome', 'This is your first note.'), "
        "(2, 'Reminder', 'Submit the SSI lab report on time.'), "
        "(3, 'Secret', 'The admin password is hunter2.')"
    )
    conn.commit()
    conn.close()
```

```

def search_notes(query):
    """Search notes by title: VULNERABLE to SQL injection."""
    conn = sqlite3.connect(DB_FILE)
    # CWE-89: string concatenation in SQL query
    sql = "SELECT id, title, body FROM notes WHERE title LIKE '%" + query
+ "%'"
    print(f"[DEBUG] Executing SQL: {sql}")
    try:
        cursor = conn.execute(sql)
        results = cursor.fetchall()
        if results:
            for row in results:
                print(f" [{row[0]}] {row[1]}: {row[2]}")
        else:
            print(" No notes found.")
    except sqlite3.Error as e:
        print(f" SQL error: {e}")
    conn.close()

def export_note(note_id):
    """Export a note to a file: VULNERABLE to command injection."""
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.execute(
        "SELECT title, body FROM notes WHERE id = ?", (note_id,)
    )
    row = cursor.fetchone()
    conn.close()

    if row is None:
        print(" Note not found.")
        return

    filename = input(" Enter filename to export to: ")
    # CWE-78: unsanitised input passed to shell
    cmd = f"echo 'Title: {row[0]}\nBody: {row[1]}' > {filename}"
    print(f"[DEBUG] Executing command: {cmd}")
    os.system(cmd)
    print(f" Note exported to {filename}")

```

```
def main():
    init_db()
    while True:
        print("\n=== Note App ===")
        print("1. Search notes")
        print("2. Export note")
        print("3. Quit")
        choice = input("Choice: ").strip()

        if choice == "1":
            query = input("  Search query: ")
            search_notes(query)
        elif choice == "2":
            try:
                note_id = int(input("  Note ID: "))
            except ValueError:
                print("  Invalid ID.")
                continue
            export_note(note_id)
        elif choice == "3":
            break
        else:
            print("  Invalid choice.")

if __name__ == "__main__":
    main()
```

Exercícios

Exercício 1: Reconhecimento de SQL injection

Execute a aplicação e use a funcionalidade de pesquisa normalmente:

```
python3 noteapp.py
```

Pesquise por `Welcome`. Agora experimente as seguintes entradas e documente o resultado de cada uma:

1. `' OR '1'='1`
2. `' UNION SELECT 1, sql, '' FROM sqlite_master --`

3. `' UNION SELECT 1, title, body FROM notes --`

No seu relatório, explique o que cada *payload* faz e porque funciona. Que informação pode um atacante extrair?

Exercício 2: Command injection

Use a funcionalidade de exportação (opção 2) para exportar a nota 1. Quando lhe for pedido um nome de ficheiro, experimente:

1. `note.txt` (uso normal)
2. `note.txt; cat /etc/passwd`
3. `note.txt; id; whoami`
4. ``ls -la``

Documente o *output* de cada. Explique por que razão a aplicação é vulnerável e o que o atacante poderia alcançar num servidor real.

Nota sobre `cat /etc/passwd` entre plataformas. Num *host* Linux (WSL2, uma VM ou uma instalação nativa), `/etc/passwd` lista contas Unix reais (`root` , `nobody` , e o utilizador Linux que criou). Se estiver a executar este trabalho Python diretamente em macOS, o mesmo ficheiro contém contas de serviço da Apple (`_trustd` , `_mmaintenanced` , e similares) em vez das entradas convencionais. O conteúdo difere, mas a implicação de segurança é idêntica: um atacante não autenticado com capacidade de *command injection* tem acesso de leitura a todos os ficheiros legíveis pelo utilizador do serviço, e qualquer dessas plataformas ficaria igualmente comprometida num servidor real.

Exercício 3: Remediação segura (SQL injection)

Modifique a função `search_notes` para usar *queries* parametrizadas. A sua correção deve:

1. Usar *placeholders* `?` em vez de concatenação de *strings*.
2. Preservar a funcionalidade de pesquisa `LIKE` com *wildcards*.
3. Garantir que nenhum dos *payloads* do Exercício 1 funciona depois da correção.

Inclua a função corrigida e demonstre que os *payloads* já não são bem-sucedidos.

Exercício 4: Remediação segura (command injection)

Modifique a função `export_note` para eliminar a vulnerabilidade de *command injection*. Deve:

1. Substituir `os.system()` por uma alternativa segura (considere: precisa sequer de uma *shell*?).
2. Validar ou sanitizar o nome do ficheiro.

3. Garantir que nenhum dos *payloads* do Exercício 2 funciona depois da correção.

Inclua a função corrigida e os resultados dos testes no seu relatório.

Exercício 5: Reflexão

Num ensaio curto (200 a 350 palavras), responda às seguintes questões:

1. Qual é a causa comum partilhada por *buffer overflows*, vulnerabilidades de *string de formato*, SQL injection e *command injection*?
2. Porque é que a validação de entradas, por si só, é insuficiente como estratégia de defesa?
3. Como é que os princípios da "parametrização" e do "privilégio mínimo" se aplicam às vulnerabilidades que explorou nos trabalhos da Semana 11 e da Semana 12?
4. Tanto os *buffer overflows* como as vulnerabilidades de *string de formato* (exploradas na Semana 11) expõem memória da *stack* a um atacante. Em que aspetos os dois mecanismos de ataque diferem quanto ao modo como o atacante lê ou manipula a memória? (Consulte as Partes A e B do enunciado da Semana 11 e os diapositivos de *Corrupção de Memória da Sessão 1 da aula, conforme necessário.*)

Referências

- CWE-89: Improper Neutralisation of Special Elements used in an SQL Command, <https://cwe.mitre.org/data/definitions/89.html>
- CWE-78: Improper Neutralisation of Special Elements used in an OS Command, <https://cwe.mitre.org/data/definitions/78.html>
- OWASP Top 10 (2025), <https://owasp.org/Top10/>

Entrega

Façam o upload dos vossos entregáveis para o repositório privado do vosso grupo em <https://github.com/uminho-lei-ssi/>, seguindo o mesmo procedimento dos trabalhos anteriores. As respostas escritas devem ser submetidas como ficheiro Markdown; o código-fonte no seu formato original.